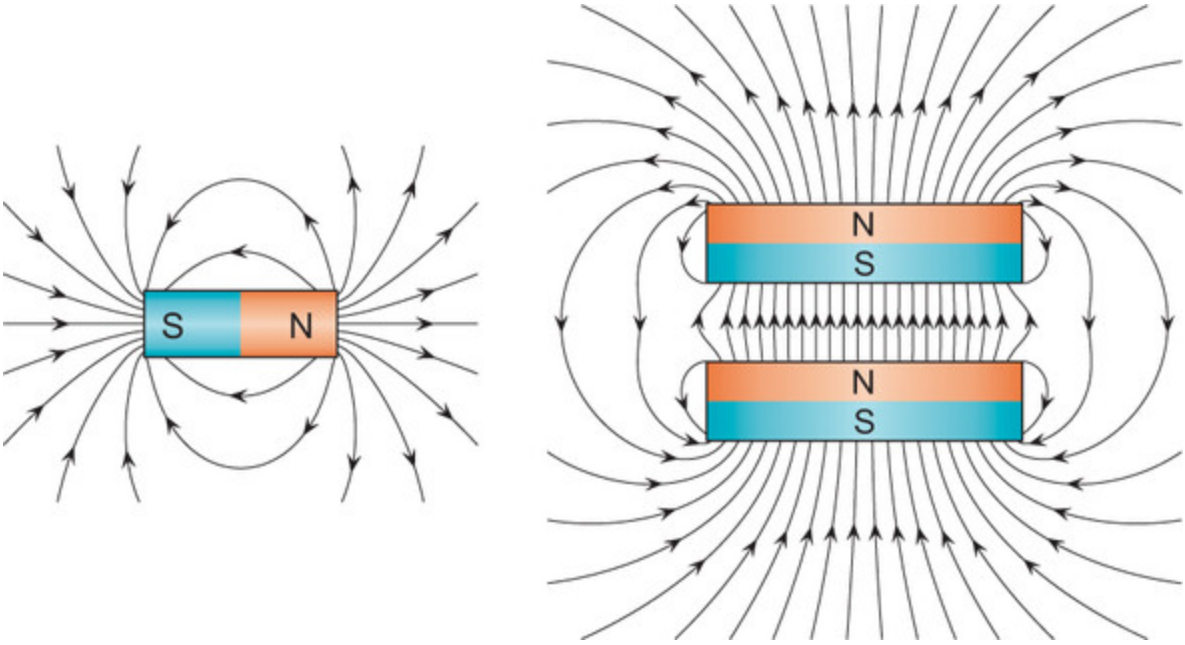
magnets with an approximation of there being one magnet with a magnetic field that is the sum of the two components. It is not the individual magnetic fields of the nearest electrons that matter so much as their sum, so rather than giving our Q function the spin information about each neighboring electron, we can instead give it the sum of their spins. For example, in the 1D grid, if the left neighbor has an action vector of [1,0] (down) and the right neighbor has an action vector of [0,1] (up), the sum would be [1,0] + [0,1] = [1,1].

Figure 9.18. Left: A single bar magnet and its magnetic field lines. Recall that a magnet has two magnetic poles, often called North (N) and South (S). Right: Put two bar magnets close together, and their combined magnetic field is a bit more complicated. When we're modeling how the electron spins behave in a 2D or 3D grid, we care about the overall magnetic field generated by the contributations of all the electrons in a neighborhood; we don't need to know what the magnetic field is for each individual electron.



Machine learning algorithms perform better when data is normalized within a fixed range like [0,1], partly due to the fact that the activation functions only output data within a limited output range (the *codomain*), and they can be "saturated" by inputs that are too large or too small. For example, the `tanh` function has a codomain (the range of values that it can possibly output) in the interval [–1,+1], so if you give it two really large but non-equal numbers, it will output numbers very close to 1. Since computers have limited precision, the output values both might end up rounding to 1